# Jaypee University of Information Technology

## Department of Computer Science and Engineering and Information Technology

**Major Project - I (18B19CI791) | AY 2025-26**

**End-Term Evaluation | December 12, 2025.**

# CollabCode Studio: Real-Time Coding & Visual Collaboration Platform.

**Group No. : 80**


**Team Member (s)**

- Divyanshu Chander (221030460)
- Kuldeep Verma (221031057)
- Dhruv Malik (221030423)
- Akshat Kapoor (221030296)

**Supervisor**

- Name: Ms. Nitika
  Designation: Assistant Professor

# Outline

- Introduction
- Problem Statement
- Objectives
- Literature Review
- Work Done (after Mid-Term Evaluation)
- Project Design
- Tools, Technologies and Languages
- Dataset
- Implementation
- Experimental Results and Evaluation
- Key Learnings
- Work Contribution and Attendance
- Supervisor Interactions
- Future Work
- Project Plan
- References

# Introduction

- **CollabCode Studio** is an integrated, web-based platform designed to streamline the software development workflow for remote and distributed teams.
- In the current development landscape, teams are forced to use a **fragmented set of tools** for coding, communication, and visual planning, leading to inefficiency and context switching.
- Our project addresses this by **unifying these core functionalities** into a single, cohesive environment.
- The platform combines a **real-time collaborative code editor**, an **integrated chat**, a **shared visual drawing board**, and **AI-powered assistance** to create a seamless and productive workspace.

# Problem Statement

- Modern software development is highly collaborative but relies on a **disconnected toolchain** (e.g., VS Code + Slack + Miro).
- This fragmentation leads to several key issues:
  - **Reduced Productivity:** Constant context-switching between applications breaks developers' focus and "flow state."
  - **Communication Gaps:** Project discussions and decisions are scattered across different platforms, making it hard to track context.
  - **Onboarding Hurdles for Learners:** New developers face a steep learning curve in setting up complex local development environments.
- There is a clear need for a unified platform that centralizes these activities, reduces friction, and provides an accessible environment for both professionals and learners.

# Objectives

- **To Develop a Unified Collaborative Platform:** Design and build an integrated web application featuring a real-time code editor, chat, and visual drawing board.
- **To Ensure Real-Time Synchronization:** Implement a robust, low-latency synchronization mechanism using WebSockets and CRDTs to provide a seamless multi-user experience.
- **To Integrate AI-Powered Assistance:** Incorporate an AI assistant to provide intelligent code suggestions and error detection, enhancing developer productivity.
- **To Create an Accessible Learning Tool:** Offer a zero-setup, browser-based environment to lower the barrier to entry for students and aspiring developers.
- **To Deploy a Scalable Cloud-Native Application:** Containerize the application using Docker and deploy it on a scalable cloud infrastructure like AWS EC2 to ensure high availability and performance.

# Literature Review

| S. No. | Author & Paper Title [Citation] | Journal/ Conference (Year) | Tools/ Techniques/ Dataset | Key Findings/ Results | Limitations/ Gaps Identified |
|---|---|---|---|---|---|
| 1. | I. David and E. Syriani, "Real-time collaborative multi-level modeling by conflict-free replicated data types" [1] | Software and Systems Modeling, 2022 | Conflict-Free Replicated Data Types (CRDTs) | Demonstrates the effectiveness of CRDTs in real-time collaborative modeling, merging concurrent edits without complex transformation logic. | Can have high memory overhead as data structures grow. |
| 2. | P. S. Almeida, "Approaches to Conflict-free Replicated Data Types" [2] | arXiv preprint arXiv:2310.18220, 2023 | CRDTs (operation-based and state-based) | Provides a comprehensive analysis of different approaches to CRDTs, reinforcing their selection as a modern solution for managing shared state. | Being a survey, it reviews existing work and does not introduce new algorithms & metrics. |
| 3. | L. Chen, et al., "A Survey on Evaluating Large Language Models in Code Generation" [3] | arXiv preprint arXiv:2408.16498, Aug. 2024 | Large Language Models (LLMs) | Offers a crucial framework for selecting and validating the effectiveness of AI models for code suggestion and error-checking features. | As a survey, it reviews existing work and does not introduce a novel evaluation method or a new LLM. |
| 4. | C. Treude and M. A. Gerosa, "How Developers Interact with AI: A Taxonomy of Human-AI Collaboration in Software Engineering" [4] | arXiv preprint arXiv:2501.08774, 2025 | AI in Software Engineering | Provides a detailed taxonomy of how developers interact with AI, offering valuable insights for designing intuitive human-AI collaboration features. | Conceptual Paper, Lacks Empirical Data to validate it's impact. |

# Literature Review (cont…)

| S. No. | Author & Paper Title [Citation] | Journal/ Conference (Year) | Tools/ Techniques/ Dataset | Key Findings/ Results | Limitations/ Gaps Identified |
|---|---|---|---|---|---|
| 5. | R. Ferdiana, "The Impact of Artificial Intelligence on Programmer Productivity" [5] | Conference Paper, 2024 | Artificial Intelligence (AI) | Highlights the positive impact of AI on programming tasks and enhancing developer productivity. | Conceptual Paper, Lacks Empirical Data to validate it's impact. |
| 6. | A. Gote, "Real-time Interactivity in Hybrid Applications With Web Sockets" [6] | International Research Journal of Modernization in Engineering Technology and Science (IRJMETS), 2024 | WebSockets | Confirms that WebSockets provide the persistent, low-latency, bidirectional communication channel necessary for a seamless collaborative experience. | It does not introduce a new protocol or provide a quantitative performance comparison against other real-time technologies like long-polling. |
| 7. | M. C. da Silva, et al., "A Visual Tool for Supporting Collaborative Code Quality" [7] | 23rd International Conference on Computer Supported Cooperative Work in Design (CSCWD), 2019 | Visual Tools for Code Quality | Addresses the need for visual tools to support collaborative coding, which is a core feature of the project. | Does not include a large-scale empirical study with industry developers to quantitatively measure its impact on productivity or code quality. |
| 8. | M. B. Khan, et al., "Design and Development of Real-time Code Editor for Collaborative Programming" [8] | International Journal of Scientific Research in Computer Science and Engineering, vol. 11, no. 6, pp. 19-26, Dec. 2023 | Real-time Code Editors | Reinforces the importance of WebSockets for a seamless collaborative experience in real-time code editors. | Does not provide a comparative analysis of other key synchronization methods like CRDTs or Operational Transformation (OT) |

# Literature Review (cont…)

| S. No. | Author & Paper Title [Citation] | Journal/ Conference (Year) | Tools/ Techniques/ Dataset | Key Findings/ Results | Limitations/ Gaps Identified |
|---|---|---|---|---|---|
| 9. | G. Jadhav and F. Gonsalves, "Role of Node.js in Modern Web Application Development" [9] | International Research Journal of Engineering and Technology (IRJET), vol. 7, no. 6, 2020 | Node.js | Details the role of Node.js in creating efficient, scalable server-side applications. | The provided performance test is based on a specific use case, not a comprehensive benchmark across various workloads. |
| 10. | R. Saini and R. Behl, "An Introduction to AWS - EC2 (Elastic Compute Cloud)" [10] | International Conference on Recent Mechanical and Automation Technology (ICRMAT), 2020 | AWS EC2 | Provides practical guidance on architecting and deploying applications on AWS, informing the project's deployment strategy. | This is a high-level introductory paper. It describes the service and its benefits but does not include a technical deep-dive. |
| 11. | B. Bachina, "Architecting and Deploying MERN Stack Applications on AWS ECS" [11] | Journal of Scientific and Engineering Research, vol. 9, no. 2, pp. 123-142, 2022 | MERN, AWS ECS | Provides practical guidance on architecting and deploying MERN applications on AWS, informing the project's deployment strategy. | Does not provide novel performance metrics or an analysis against alternative deployment platforms. |
| 12. | I. Yusron and A. Wibowo, "A Performance Analyst Comparison of ReactJS and AngularJS in the Front-End Website" [12] | International Journal of Science and Applied Information Technology (IJSait), 2020 | ReactJS, AngularJS | Analyzes the performance benefits of React for building interactive user interfaces. | The study is based on a single application and compares ReactJS to AngularJS (the older version), not the more modern Angular framework. |

# Literature Review (cont…)

| S. No. | Author & Paper Title [Citation] | Journal/ Conference (Year) | Tools/ Techniques/ Dataset | Key Findings/ Results | Limitations/ Gaps Identified |
|---|---|---|---|---|---|
| 13. | M. Kotsovoulou and V. Stefanou, "Student Perceptions on the Effectiveness of Collaborative Problem-based Learning Using Online Pair Programming Tools" [13] | WSEAS Transactions on Proof, vol. 1, pp. 15-22, 2021 | Online Pair Programming Tools | A qualitative study that found students perceive these tools as highly effective for problem-solving, increasing participation, and enhancing engagement and knowledge sharing. | This is a qualitative study focused on student perceptions and does not provide quantitative data. |
| 14. | J. Fiala, M. Yee-King, and M. Grierson, "Collaborative coding interfaces on the Web" [14] | International Conference on Live Coding (ICLI), 2016 | Web-based Collaborative Coding | It explores the design and implementation of collaborative coding interfaces on the web, relevant to the project's core functionality. | Collaboration is primarily text-based (editor); lacks explicit integration of a shared visual brainstorming canvas or integrated AI features. |
| 15. | S. W. Lee, et al., Communication, Control, and State Sharing in Networked Collaborative Live Coding (UrMus)" [15] | ACM Computing Surveys (CSUR), vol. 54, no. 1, pp. 1-38, 2021 | Real-time Collaborative Editing Systems | Provides a comprehensive overview of the field, which is foundational to the project. | Focuses on Live Music/Audio coding (Lua/UrMus); the solutions are not generalized to the broader general software development or visual collaboration needs. |

# Work Done (after Mid-Term Evaluation)

## October 2025 (System Design Finalization)

- Finalize the detailed System Architecture, Component Diagrams, and Sequence Diagrams.
- Review libraries, tools & frameworks and decide possible approach.

## November 2025 (Feature Development)

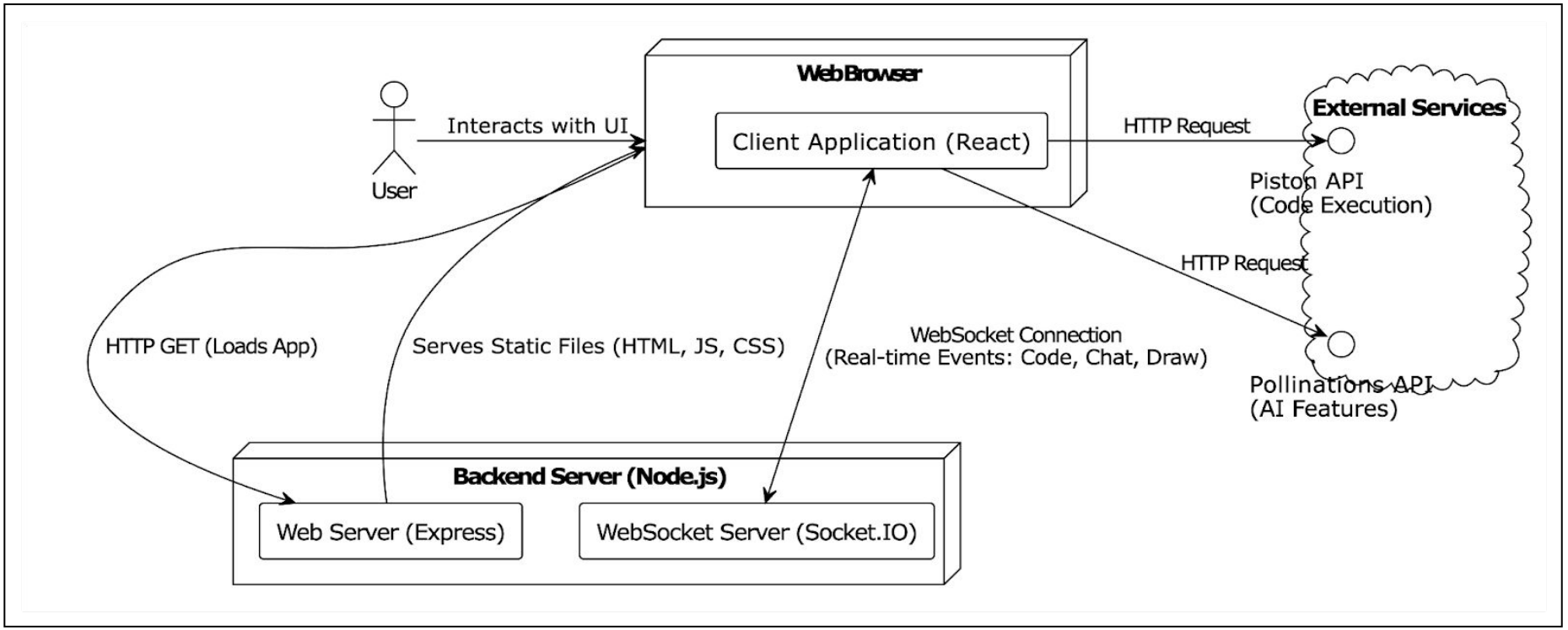
- Implement the basic frontend structure with ReactJS, including routing and main views.
- Implement the Real-Time Chat functionality..
- Start working on Real-Time Collaborative Editor synchronization.

## December 2025 (Integration, Testing & Evaluation Preparation)

- Integrating and Testing the MVP features like (editor & chat).
- Prepare documentation, project presentation & demo for evaluation.

# Project Design

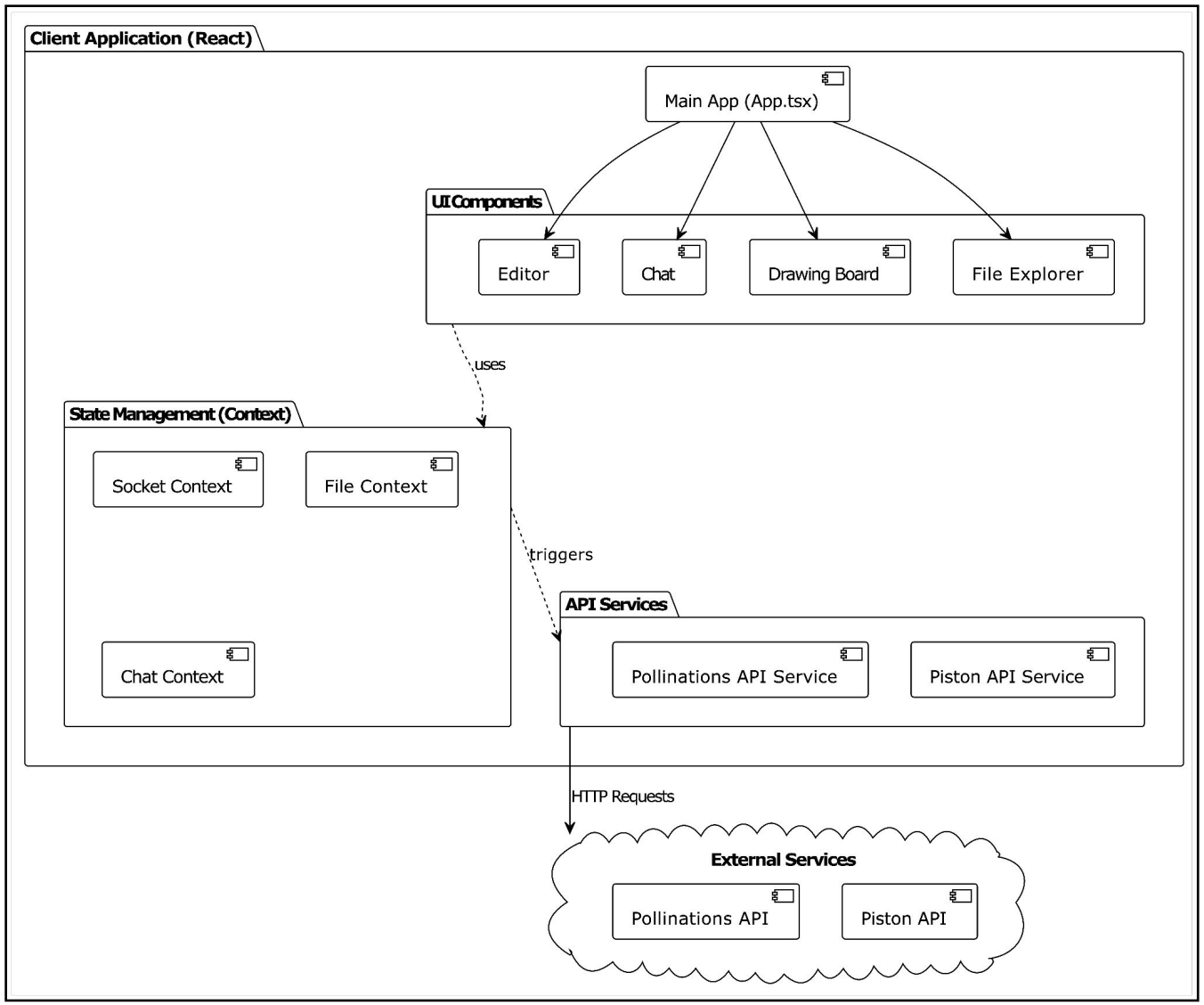
- **High Level System Architecture**




Shows the **overall interaction** between the Client Application, the Backend Server, and External Services. It highlights how the client communicates with the backend via WebSockets and HTTP while also making direct API calls to third-party services.

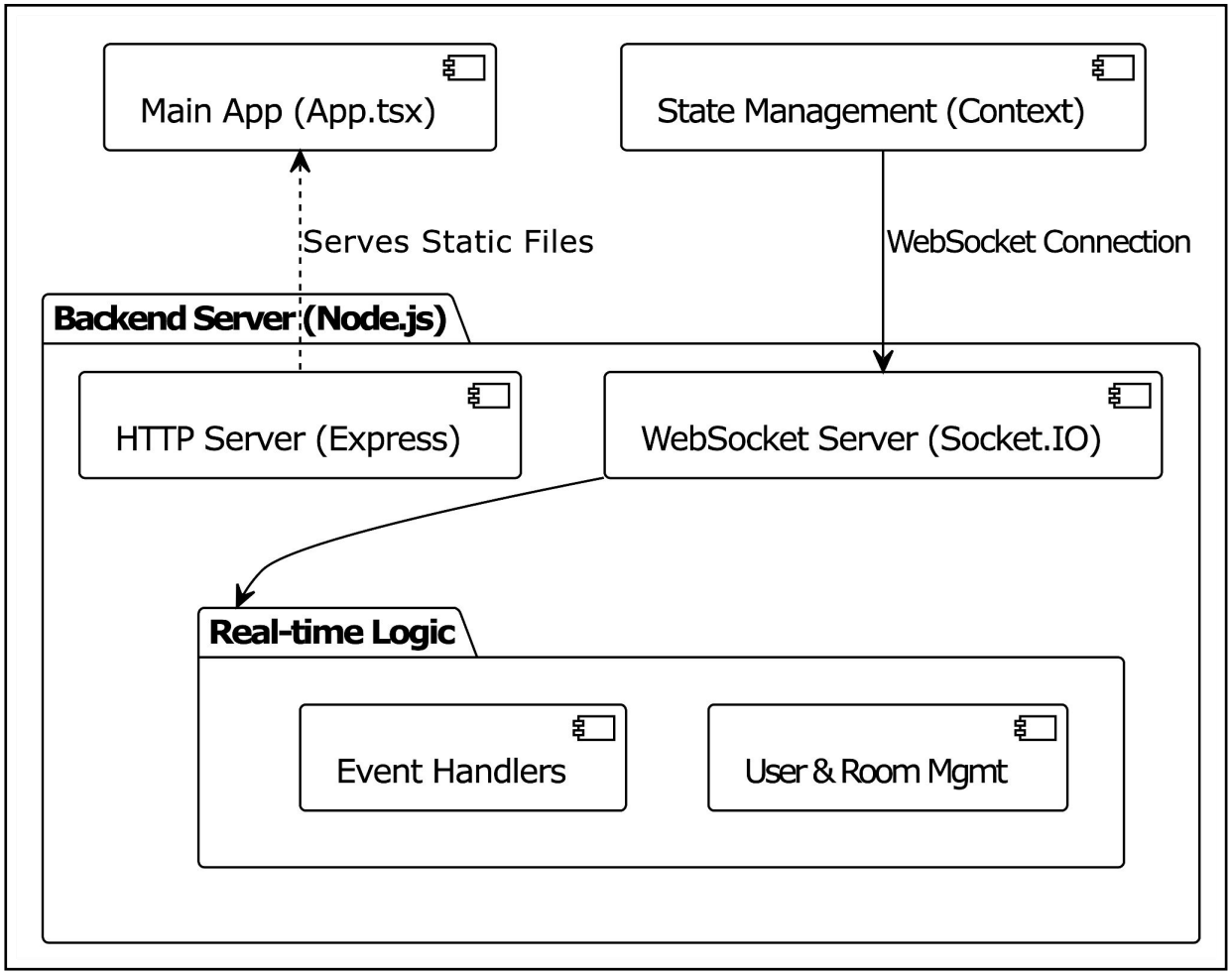
# Project Design (cont…)

- **Client Application Diagram**



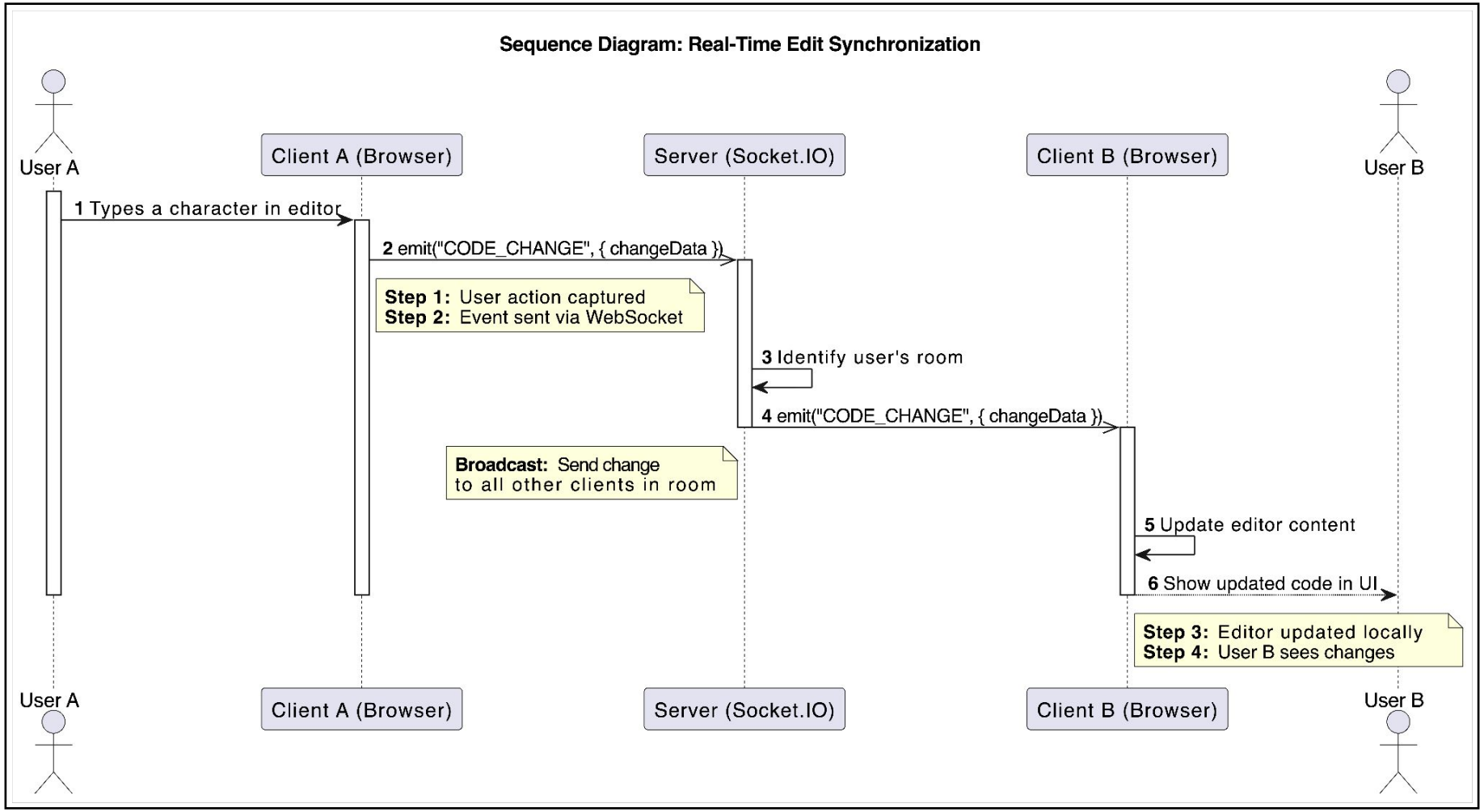
# Project Design (cont…)

- **Backend Server Diagram**



# Project Design (cont…)

- **Sequence Diagram – Real-Time Edit Synchronisation**



# Tools, Technologies and Languages

## Languages

- **JavaScript (ES6+):** The primary programming language for both frontend and backend.
- **TypeScript:** A statically typed superset of JavaScript for enhanced code quality and maintainability.

## Frontend Development

- **ReactJS (v18.x):** Core UI library for building a component-based, single-page application.
- **Vite (v5.x):** A modern, high-performance build tool for a rapid development workflow.
- **Tailwind CSS (v3.x):** A utility-first CSS framework for efficient and responsive styling.

# Tools, Technologies and Languages (cont…)

**Backend Development**

- **Node.js (v20.x LTS):** The JavaScript runtime environment for the server.
- **Express.js (v4.x):** A minimal web application framework for building the REST API.
- **Socket.IO (v4.x):** The core library for enabling real-time, bidirectional WebSocket communication.

**Deployment & DevOps**

- **Docker (v24.x):** Used for containerizing the application to ensure a consistent and isolated environment.
- **AWS EC2:** The cloud platform for deploying and hosting the scalable backend server.

**General Tools**

- **Git & GitHub:** For version control and collaborative code management.

# Dataset

# Implementation

- **Project Directory Structure**

```
Collab Code Studio/
│
├── client/                          # React Frontend Application
│   ├── src/
│   │   ├── components/              # UI Components
│   │   │   ├── editor/              # Code editor components
│   │   │   ├── files/               # File tree components
│   │   │   ├── sidebar/             # Sidebar & panels
│   │   │   ├── chats/               # Chat functionality
│   │   │   ├── drawing/             # Whiteboard
│   │   │   ├── run/                 # Code execution UI
│   │   │   └── landing/             # Homepage forms
│   │   │
│   │   ├── context/                 # State Management (9 providers)
│   │   │   ├── FileContext.tsx      # File system ⭐
│   │   │   ├── SocketContext.tsx    # WebSocket ⭐
│   │   │   ├── CopilotContext.tsx   # AI integration
│   │   │   ├── RunCodeContext.tsx   # Code execution
│   │   │   └── ...                  # Other contexts
│   │   │
│   │   ├── pages/                   # Route pages
│   │   │   ├── HomePage.tsx
│   │   │   └── EditorPage.tsx       # Main editor ⭐
│   │   │
│   │   ├── types/                   # TypeScript definitions
│   │   ├── hooks/                   # Custom React hooks
│   │   ├── utils/                   # Helper functions
│   │   └── resources/               # Themes & assets
│   │
│   ├── package.json
│   ├── vite.config.ts
│   └── tsconfig.json
│
├── server/                          # Node.js Backend
│   ├── src/
│   │   ├── server.ts                # Socket.IO server ⭐
│   │   └── types/                   # Shared TypeScript types
│   │
│   ├── package.json
│   └── tsconfig.json
```

# Implementation

- **Server Initialization & Configuration**

```
const app = express();
app.use(express.json());
app.use(cors());
app.use(express.static(path.join(__dirname, "public")));

const server = http.createServer(app);
const io = new Server(server, {
  cors: {
    origin: "*",  // Allow all origins for collaboration
  },
  maxHttpBufferSize: 1e8,  // 100MB buffer for large file transfers
  pingTimeout: 60000,      // 60s timeout to handle slow connections
});

let userSocketMap: User[] = [];  // Global state for active users
```

- **Realtime File Synchronisation**

```
// When user joins, sync complete file structure
const handleUserJoined = useCallback(({ user }: { user: RemoteUser }) => {
    toast.success(`${user.username} joined`)

    // Send complete state to new user's socket
    socket.emit(SocketEvent.SYNC_FILE_STRUCTURE, {
        fileStructure,    // Entire directory tree
        openFiles,        // Currently open files
        activeFile,       // Active editor file
        socketId: user.socketId  // Target specific user
    })

    setUsers((prev) => [...prev, user])
}, [fileStructure, openFiles, activeFile, socket])
```

# Implementation

- **Tooltip for real time cursor**

```
// CodeMirror StateField for real-time cursor awareness
export const tooltipField = (users: User[]) => {
    return StateField.define<readonly Tooltip[]>({
        create: getCursorTooltips(users),

        update(tooltips, tr) {
            if (!tr.docChanged && !tr.selection) return tooltips
            return getCursorTooltips(users)(tr.state)
        },

        provide: (f) => showTooltip.computeN([f], (state) =>
            state.field(f)
        ),
    })
}

function getCursorTooltips(users: User[]) {
    return (state: EditorState): readonly Tooltip[] => {
        return users.filter(u => u.typing).map(user => ({
            pos: user.cursorPosition,
            above: true,
            create: () => {
                const dom = document.createElement("div")
                dom.className = "cm-cursor-tooltip"
                dom.textContent = user.username
                return { dom }
            }
        }))
    }
}
```

# Implementation

- **Websockets Event Handler**

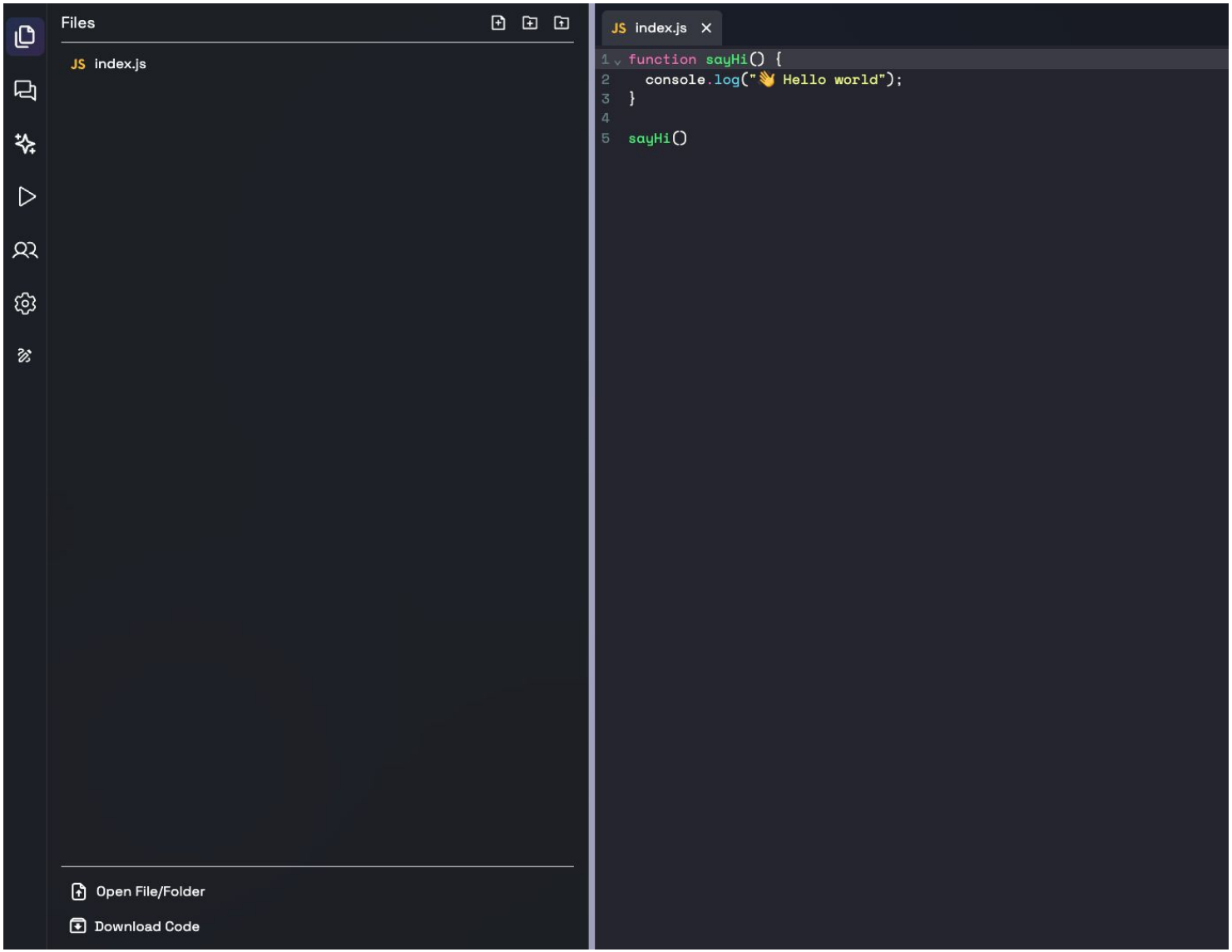
```
// Server-side collaboration logic
io.on("connection", (socket) => {

    // User joins room
    socket.on(SocketEvent.JOIN_REQUEST, ({ roomId, username }) => {
        // Check duplicate username
        const isUsernameExist = getUsersInRoom(roomId)
            .filter(u => u.username === username)

        if (isUsernameExist.length > 0) {
            socket.emit(SocketEvent.USERNAME_EXISTS)
            return
        }

        // Create user session
        const user = { username, roomId, socketId: socket.id,
                       cursorPosition: 0, typing: false }
        userSocketMap.push(user)
        socket.join(roomId)

        // Broadcast to room
        socket.broadcast.to(roomId)
            .emit(SocketEvent.USER_JOINED, { user })

        // Send existing users to new joiner
        socket.emit(SocketEvent.JOIN_ACCEPTED, {
            user,
            users: getUsersInRoom(roomId)
        })
    })

    // File operations broadcast
    socket.on(SocketEvent.FILE_UPDATED, ({ fileId, newContent }) => {
        const roomId = getRoomId(socket.id)
        socket.broadcast.to(roomId)
            .emit(SocketEvent.FILE_UPDATED, { fileId, newContent })
    })
})
```

# Implementation

- **Room Creation & Joining Page**

# Implementation

- **Editor Layout**



# Experimental Results and Evaluation

- **Functional Completion**

1. **Real-Time Collaborative Editor**
    a. Many people can edit the same file at the same time.
    b. Socket.IO keeps edits in sync in real time with no inconsistency.
    c. All connected users receive reflections of cursor positions and typing indicators.
2. **Project Structure Synchronization**
    a. It sends the entire file and folder structure to any new user who joins a session.
    b. Conflicts of duplicate file names are resolved automatically.
3. **Integrated Live Chat**
    a. Seamless conversation in a room, with no third-party apps necessary.
4. **Interactive Whiteboard**
    a. Multi-user real-time drawing with stroke-level synchronization.
    b. Smooth loading of the existing canvas state when new users join.
5. **AI Powered Features**
    a. Snippets are generated and code recommended through API calls.

# Experimental Results and Evaluation (cont…)

- **Comparison with Existing Solutions**

| Feature/Criteria | VS Code Live Share | Replit Multiplayer | Google Colab | CollabCode Studio |
|---|---|---|---|---|
| **Real-Time Code Editing** | Yes | Yes | Limited | Yes(via Socket.IO) |
| **Built-in Chat** | No | Limited | Yes | Yes |
| **Shared Visual Board** | No | No | No | Yes (Real-Time, Multi-user) |
| **AI Code Suggestion** | Yes(via extension) | Yes | Yes | Yes (Integrated) |
| **Multi-Language Support** | Yes | Yes | Python Only | Yes (via Piston API) |
| **File Structure Sync** | Limited | Yes | No | Yes |
| **Setup Effort** | High (requires VS Code) | Medium | Low | Very Low (Browser-based) |
| **One-Stop Collaboration** | Partial | Partial | Partial | Fully Unified Workspace |

# Key Learnings

**Learning Outcome:** Gained a deep, practical understanding of full-stack web development and the complexities of building real-time, distributed systems from the ground up.

**Methodology Followed:**

- **Literature Survey:** Studied foundational research papers on CRDTs, WebSockets, and AI in software engineering to inform our technical decisions.

**Tools Learned:**

- **Frontend:** ReactJS, Vite, Tailwind CSS
- **Backend:** Node.js, Express.js
- **Real-Time Communication:** Socket.IO
- **Deployment:** Docker, AWS EC2

# Work Contribution and Attendance

**GitHub Repository URL:** https://github.com/divyanshuchander/CollabCode-MajorProject.git

| Team Member | Roll No. | Work Done (provide complete details) | Work Contribution (%) | Lines of Code (if any) | Lab Attendance (%) |
|---|---|---|---|---|---|
| 1. | 221030460 | • Preparation of project proposal.<br>• Maintained the git repository<br>• Ideated the project's system architecture design.<br>• Created system architecture flowcharts.<br>• Prepared initial draft of project report.<br>• Worked on presentation<br>• Implemented the Initial Project MVP | 35% | ~1500 | 100% |
| 2. | 221030423 | • Ideated the project's system architecture design.<br>• Prepared initial draft of project report.<br>• Worked on end-term presentation. | 15% | - | 100% |
| 3. | 221031057 | • Prepared initial draft of project report.<br>• Worked on end-term presentation,<br>• Literature review of research papers<br>• Designed the Frontend UI and integrated the Code Editor.<br>• Designed a modular component-based architecture to ensure the frontend is scalable and easy to maintain.<br>• Diary Management | 35% | ~600 | 100% |
| 4. | 221030296 | • Worked on end-term presentation,<br>• Literature review of research papers<br>• Diary Management | 15% | - | 100% |

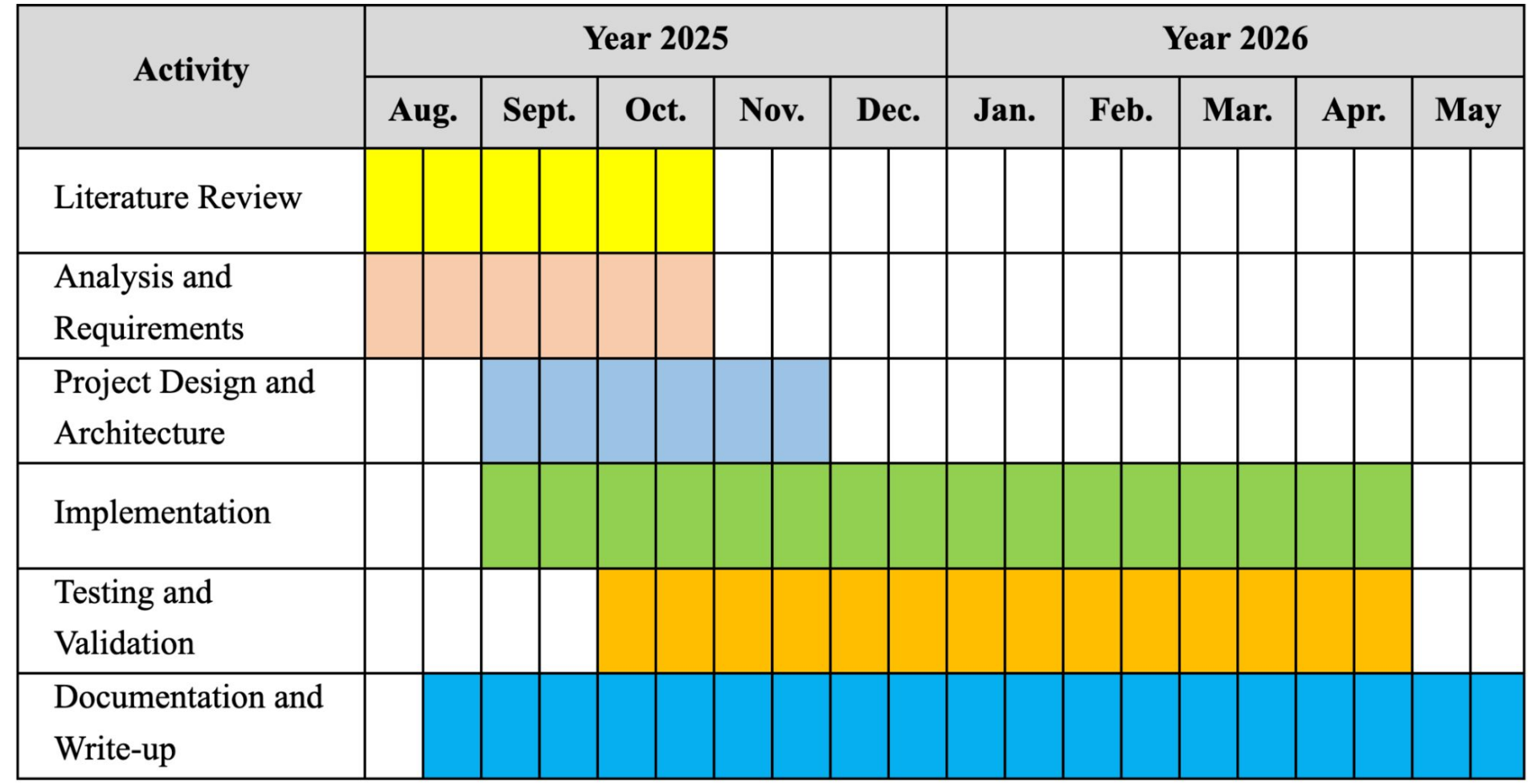
# **Supervisor Interactions** (as mentioned in weekly log)

| **No. of Meetings with Supervisor:** | | | |
|---|---|---|---|
| Week No. | Duration | Remarks (**as mentioned in the weekly log**) | Incorporated (Yes/No) |
| 1. | 25/08/2025 to 01/09/2025 | • Improvements in proposal<br>• Suggestions on the project and its approach. | Yes |
| 2. | 15/09/2025 to 22/09/2025 | • Read recent papers.<br>• Improve format of literature survey. | Yes |
| 3. | 22/09/2025 to 29/09/2025 | • Improve formatting of initial project report draft.<br>• Corrections in the Mid-Term Evaluation presentation. | Yes |
| 4. | 3/10/2025 to 9/10/2025 | • Approved the system architecture.<br>• Start the implementation of the core modules. | Yes |
| 5. | 25/10/2025 to 31/10/2025 | • Demonstrate the initial prototype.<br>• Focus on improving the User Interface (UI). | Yes |

# **Supervisor Interactions** (as mentioned in weekly log)

| **No. of Meetings with Supervisor:** | | | |
|---|---|---|---|
| Week No. | Duration | Remarks (**as mentioned in the weekly log**) | Incorporated (Yes/No) |
| 6. | 1/11/2025 to 7/11/2025 | • Ensure proper error handling in the code.<br>• Update the weekly progress log regularly. | Yes |
| 7. | 10/11/2025 to 15/11/2025 | • Start testing the application for bugs.<br>• Document the test cases and results. | Yes |
| 8. | 17/11/2025 to 22/11/2025 | • Work on the Final Report formatting. | Yes |
| 9. | 24/11/2025 to 29/11/2025 | • Correction in the Formatting of the report<br>• Prepare for the final presentation. | Yes |
| 10. | 24/11/2025 to 29/11/2025 | • Project work satisfactory.<br>• Submit the final report and code. | Yes |

# Project Plan



# Future Work

1. **Persistent Storage :** Implement database integration (MongoDB) for saving workspaces and current progress of the project.
2. **User Authentication :** Addition of user authentication system for personalised workspace management providing the ability to access different project rooms. Also adding room hierarchy among team members and adding editing, viewing, etc rights.
3. **Addition of Customisation features :** Features such as custom themes, fonts, etc can be added.
4. **Package Management & Dependencies :**
    a. Support for different package managers for development like pip, npm, Maven, etc.
    b. Addition of Unix-based filesystem to provide ability to use terminal, install dependencies, libraries and setup the system.
5. **Performance & Scalability:**
    a. Implement robust algorithms like CRDT algorithm for a better conflict-free editing.
    b. Using WebRTC for peer-to-peer communication to decrease server load.
6. **Enhanced AI Capabilities:** Context-Aware Intelligent code completion & bugs detection during development.

# References


[1] I. David and E. Syriani, "Real-time collaborative multi-level modeling by conflict-free replicated data types," Software and Systems Modeling, 2022. [Online]. Available: https://experts.mcmaster.ca/display/publication3281463

[2] P. S. Almeida, "Approaches to Conflict-free Replicated Data Types," arXiv preprint arXiv:2310.18220, 2023. [Online]. Available: https://arxiv.org/html/2310.18220v2

[3] L. Chen, et al., "A Survey on Evaluating Large Language Models in Code Generation," arXiv preprint arXiv:2408.16498, Aug. 2024. [Online]. Available: https://www.researchgate.net/publication/383529947_A_Survey_on_Evaluating_Large_Language_Models_in_Code_Generation_Tasks

[4] C. Treude and M. A. Gerosa, "How Developers Interact with AI: A Taxonomy of Human-AI Collaboration in Software Engineering," arXiv preprint arXiv:2501.08774, 2025. [Online]. Available: https://arxiv.org/pdf/2501.08774

[5] R. Ferdiana, "The Impact of Artificial Intelligence on Programmer Productivity," in Conference Paper, 2024. [Online]. Available: https://www.researchgate.net/publication/394745464_The_Impact_of_Artificial_Intelligence_on_Programmer_Jobs_Threat_or_Opportunity

[6] A. Gote, "Real-time Interactivity in Hybrid Applications With Web Sockets," International Research Journal of Modernization in Engineering Technology and Science (IRJMETS), 2024. [Online]. Available: https://www.researchgate.net/publication/378871302_Real-time_Interactivity_in_Hybrid_Applications_With_Web_Sockets

# References (cont…)


[7] M. C. da Silva, et al., "A Visual Tool for Supporting Collaborative Code Quality," in Proc. 23rd International Conference on Computer Supported Cooperative Work in Design (CSCWD), 2019. [Online]. Available: https://www.researchgate.net/publication/333516254_A_Visual_Tool_for_Supporting_Collaborative_Code_Quality

[8] M. B. Khan, et al., "Design and Development of Real-time Code Editor for Collaborative Programming," International Journal of Scientific Research in Computer Science and Engineering, vol. 11, no. 6, pp. 19-26, Dec. 2023. [Online]. Available: https://isroset.org/journal/IJSRCSE/full_paper_view.php?paper_id=2533

[9] G. Jadhav and F. Gonsalves, "Role of Node.js in Modern Web Application Development," International Research Journal of Engineering and Technology (IRJET), vol. 7, no. 6, 2020. [Online]. Available: https://www.irjet.net/archives/V7/i6/IRJET-V7I61149.pdf

[10] R. Saini and R. Behl, "An Introduction to AWS - EC2 (Elastic Compute Cloud)," in Proc. International Conference on Recent Mechanical and Automation Technology (ICRMAT), 2020. [Online]. Available: https://www.researchgate.net/publication/348638365_An_Introduction_to_AWS-EC2_Elastic_Compute_Cloud

[11] B. Bachina, "Architecting and Deploying MERN Stack Applications on AWS ECS," Journal of Scientific and Engineering Research, vol. 9, no. 2, pp. 123-142, 2022. [Online]. Available: https://jsaer.com/download/vol-9-iss-2-2022/JSAER2022-9-2-123-142.pdf

# References (cont…)


[12] I. Yusron and A. Wibowo, "A Performance Analyst Comparison of ReactJS and AngularJS in the Front-End Website," International Journal of Science and Applied Information Technology (IJSait), 2020. [Online]. Available: https://www.semanticscholar.org/paper/Research-and-Analysis-of-the-Front-end-Frameworks-Xing-Huang/7a252edb5c9f003aee8400c109b778a350ee1411

[13] M. Kotsovoulou and V. Stefanou, "Student Perceptions on the Effectiveness of Collaborative Problem-based Learning Using Online Pair Programming Tools," WSEAS Transactions on Proof, vol. 1, pp. 15-22, 2021. [Online]. Available: https://wseas.com/journals/articles.php?id=5463

[14] J. Fiala, M. Yee-King, and M. Grierson, "Collaborative coding interfaces on the Web," in Proc. International Conference on Live Coding (ICLI), 2016. [Online]. Available: https://www.google.com/search?q=https://iclc.livecodenetwork.org/2016/papers/fiala-et-al-2016.pdf

[15] S. W. Lee and G. Essl, "Communication, Control, and State Sharing in Networked Collaborative Live Coding," in Proc. International Conference on New Interfaces for Musical Expression (NIME), 2014. [Online]. Available: https://www.nime.org/proceedings/2014/nime2014_554.pdf

[16] OpenAI, ChatGPT: GPT-5 Model, OpenAI, San Francisco, CA, USA. [Online]. Available: https://chat.openai.com

[17] Microsoft, Visual Studio Code (VS Code), Microsoft Corporation, Redmond, WA, USA. [Online]. Available: https://code.visualstudio.com

# References (cont…)

[18] Google, Gemini: Google DeepMind's Multimodal AI, Google LLC, Mountain View, CA, USA. [Online]. Available: https://deepmind.google/technologies/gemini

[19] Node.js Foundation, Node.js JavaScript Runtime Environment, OpenJS Foundation. [Online]. Available: https://nodejs.org

[20] Meta Platforms, Inc., React: A JavaScript Library for Building User Interfaces, Meta, Menlo Park, CA, USA. [Online]. Available: https://react.dev

[21] Socket.IO, Socket.IO: Real-Time Bidirectional Event-Based Communication, OpenJS Foundation. [Online]. Available: https://socket.io

Thanks.